Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche linéaire et logarithmique et analyse de complexité temporelle

Synthèse exécutive : Ce cours traite des deux méthodes fondamentales de recherche d'une valeur dans un tableau : la recherche séquentielle (ou linéaire) et la recherche dichotomique (fondée sur le paradigme 'diviser pour régner'). L'objectif est de comprendre l'impact direct de la structure et du pré-traitement des données (tableau trié ou non trié) sur la complexité algorithmique. Si la recherche séquentielle s'exécute de manière universelle en O(n) sans précondition, la recherche dichotomique exige un tableau strictement trié au préalable pour atteindre une complexité logarithmique O(log n). Le cours détaille la manipulation dynamique des pointeurs d'indices (début, fin, milieu), les critères d'arbitrage économique entre tri et recherche, ainsi que la traduction algorithmique en C++ selon les normes rigoureuses de l'IUT d'Anglet.

La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est l'algorithme de recherche le plus intuitif et universel. Il consiste à examiner une à une chaque cellule du tableau à l'aide d'une boucle, de l'indice 0 jusqu me à l'indice n-1. Pour chaque élément, l'algorithme effectue une comparaison logique avec la valeur recherchée (la cible). Deux issues sont possibles : soit une correspondance est trouvée et l'algorithme renvoie aussitôt l'indice de la cellule (succès), soit la fin du tableau est atteinte sans résultat et l'algorithme renvoie une valeur conventionnelle signifiant l'échec (généralement -1). Au pire des cas, c'est-à-dire si le tableau contient n éléments et que la cible se trouve à l'indice n-1 ou n'existe pas, l'algorithme doit réaliser exactement n comparaisons. La complexité est donc notée O(n). Cette méthode est le seul choix possible lorsque les données sont totalement désordonnées et qu'effectuer un tri préalable coûterait plus cher que la recherche elle-même.

- Parcours exhaustif et séquentiel du tableau du premier indice (0) au dernier indice (n-1)
- Absence totale de précondition : utilisable indifféremment sur tableau trié ou non trié
- Complexité temporelle linéaire O(n) au pire des cas (élément situé en toute fin ou absent)
- Arrêt anticipé immédiat dès que l'élément cible est trouvé dans la structure

Cas concret enseignant : *Le professeur a utilisé l'analogie d'une clé tombée dans un tas de feuilles mortes : le tas n'ayant aucune organisation logique ou spatiale, il est absolument obligatoire de soulever chaque feuille une par une, de la première jusqu me à la dernière, jusqu me à tomber sur la clé.*

La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite la propriété de tri d'un tableau pour éliminer massivement des sous-ensembles d'éléments sans avoir à les lire. L'algorithme délimite la fenêtre de recherche active grâce à deux pointeurs d'indices : 'debut' (initialisé à 0) et 'fin' (initialisé à n-1). À chaque étape de la boucle, on calcule l'indice central par division entière : milieu = (debut + fin) / 2. On compare ensuite le contenu de la case T[milieu] avec la valeur cible recherchée. Trois cas de figure se présentent : 1) Si T[milieu] est égal à la cible, la recherche s'arrête avec succès. 2) Si la cible est strictement inférieure à T[milieu], alors l'élément ne peut résider que dans le sous-tableau de gauche ; on restreint donc la recherche en ajustant 'fin = milieu - 1'. 3) Si la cible est strictement supérieure à T[milieu], on ajuste 'debut = milieu + 1'. Le processus s'itère tant que debut <= fin. En divisant l'espace de recherche par deux à chaque étape, un tableau de 1 000 000 d'éléments est exploré en seulement 20 comparaisons au maximum (car 2^20 ≈ 1 048 576), ce qui correspond à une complexité temporelle en O(log2 n).

- Précondition stricte et obligatoire : le tableau doit être trié au préalable
- Principe du Diviser pour Régner : réduction de 50 % de l'espace de recherche à chaque itération
- Gestion dynamique de trois pointeurs d'indices : debut, fin et milieu = (debut + fin) / 2

• Complexité temporelle logarithmique O(log2 n) au pire des cas

Cas concret enseignant : *Le professeur a illustré cela avec le jeu du 'Devine un nombre entre 1 et 100' : proposer 1, 2, 3... est une stratégie séquentielle médiocre. Proposer 50 élimine d'un coup la moitié des possibilités selon que la réponse est 'c'est plus grand' ou 'c'est plus petit'. En poursuivant cette dichotomie, la réponse est garantie en 7 étapes maximum (2^7 = 128 > 100).*

Arbitrage et Analyse Comparative des Performances

Le choix entre recherche séquentielle et dichotomique dépend de l'état initial de la structure et du nombre d'opérations de recherche prévues. Même si la recherche dichotomique présente une complexité logarithmique extrêmement favorable O(log n), elle nécessite qu'un tri préalable soit effectué si le tableau n'est pas ordonné. Les meilleurs algorithmes de tri (Tri Rapide, Tri Fusion) s'exécutent en O(n log n). Si un programme ne doit effectuer qu'une seule et unique recherche sur un tableau non trié de n éléments, appliquer directement une recherche séquentielle O(n) est plus performant que d'exécuter un tri suivi d'une recherche dichotomique (O(n log n + log n)). En revanche, si le tableau est statique et doit faire l'objet de très nombreuses requêtes de recherche, trier le tableau une fois pour toutes au démarrage constitue un investissement algorithmique hautement rentable.

• Coût algorithmique du tri préalable : au minimum O(n log n) pour les meilleurs algorithmes de tri

• Règle de la recherche unique : la recherche séquentielle O(n) est plus efficace sur un tableau non trié

• Règle des recherches multiples : l'investissement initial du tri O(n log n) est largement amorti par n recherches dichotomiques O(log n)

Cas concret enseignant : *Le professeur a donné l'analogie du dictionnaire imprimé : ordonner l'intégralité des mots du dictionnaire par ordre alphabétique demande un effort considérable de la part de l'imprimeur (le tri initial), mais cet effort unique permet ensuite à des millions d'utilisateurs de retrouver un mot en quelques secondes pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique puisqu'elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique impose une précondition absolue : le tableau doit être trié. Si le tableau n'est pas trié, il faut le trier au préalable, ce qui coûte O(n log n) opérations. Pour une seule recherche isolée sur des données non triées, effectuer une recherche séquentielle en O(n) reste plus rapide et évite le coût du tri.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : En informatique et en algorithmique, on utilise la division entière (opérateur '/' sur des entiers en C++ ou '//' en Python). Par exemple, pour un sous-tableau allant de l'indice 0 à l'indice 9 (10 éléments, nombre pair), (0 + 9) / 2 donne 9 / 2 = 4 en division entière. L'indice milieu retenu sera donc l'indice 4.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T de n = 10 éléments entiers : T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] délimité par les indices 0 à 9. Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur cible 42. Fournir également la fonction C++ conforme aux normes d'apprentissage de l'IUT d'Anglet.

Démarche de résolution :
Méthode d'exécution pas à pas pour rechercher 42 dans T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] : - Initialisation : debut = 0, fin = 9, pos = -1. - Iteration 1 : * Calcul du milieu : milieu = (0 + 9) / 2 = 4. * Element examiné : T[4] = 19. * Comparaison : 42 > 19. La cible est plus grande, elle se situe donc dans la partie droite. * Mise à jour : debut = milieu + 1 = 5. - Iteration 2 : * Zone de recherche : debut = 5, fin = 9. * Calcul du milieu : milieu = (5 + 9) / 2 = 7. * Element examiné : T[7] = 42. * Comparaison : T[7] == 42. Égalité constatée, pos = 7. - Fin de la boucle : la condition (pos == -1) n'est plus vérifiée.

```cpp
/*
  Fichier : recherche_dichotomique.cpp
  Auteur  : Matéi de Saint Leger
  Date    : 11/11/2026
  Rôle    : Recherche dichotomique d'un entier dans un tableau trié
*/

#include <iostream>
#include <vector>
using namespace std;

// Fonction de recherche dichotomique
int rechercherDichotomique(const vector<int>& tab, int valCible)
{
    // CONSTANTES
    // (aucune constante locale requise)

    // VARIABLES
    int debut;  // Indice de début de zone
    int fin;    // Indice de fin de zone
    int milieu; // Indice central calculé
    int pos;    // Indice résultat (-1 si non trouvé)

    // TRAITEMENTS
    debut = 0;
    fin = static_cast<int>(tab.size()) - 1;
    pos = -1;

    while ((debut <= fin) && (pos == -1))
    {
        milieu = (debut + fin) / 2;
        if (tab[milieu] == valCible)
        {
            pos = milieu;
        }
        else if (valCible < tab[milieu])
        {
            fin = milieu - 1;
        }
        else
        {
            debut = milieu + 1;
        }
    }

    return pos;
}
```

Résultat : La valeur 42 a été localisée à l'indice 7 du tableau en seulement 2 comparaisons grâce à la recherche dichotomique, alors qu'une recherche séquentielle aurait demandé 8 comparaisons (de T[0]=3 jusqu me à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la précondition indispensable et obligatoire pour pouvoir exécuter une recherche dichotomique sur un tableau ?

R : Le tableau doit obligatoirement être préalablement trié (dans l'ordre croissant ou décroissant).

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : La complexité temporelle est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : Elle nécessite au maximum 11 comparaisons au pire des cas (car 2^10 = 1024 et log2(1024) = 10 itérations + 1 comparaison finale).